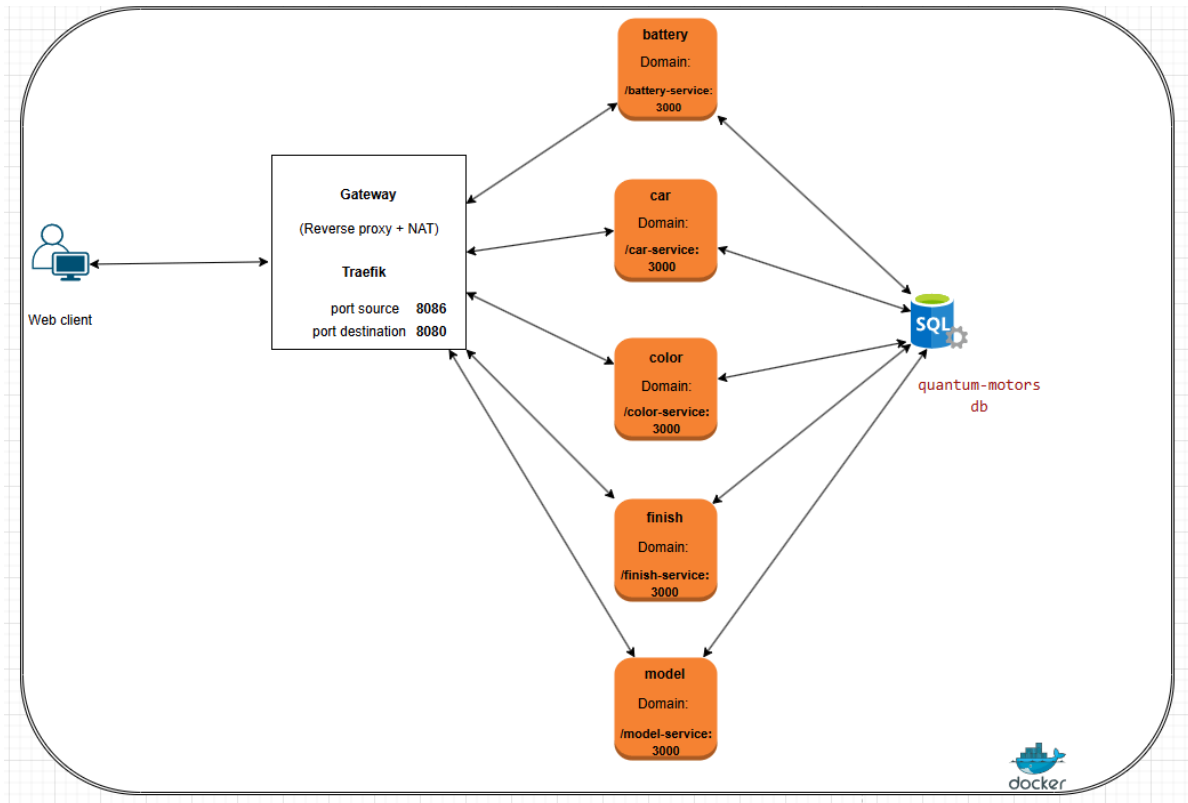


architecture microservices



L'architecture microservices illustrée ci-dessus est structurée autour d'une série de services spécialisés pour l'application Quantum Motors.

Chaque service est conçu pour répondre à un domaine spécifique de l'application, comme la gestion des batteries, des voitures, des couleurs, des finitions et des modèles de véhicules.

Services Spécialisés : Les microservices sont décomposés en plusieurs composants indépendants :

Battery Service (/battery-service), Car Service (/car-service), Color Service (/color-service), Finish Service (/finish-service), et Model Service (/model-service).

Chaque service tourne sur son propre port interne (3000) et est responsable d'une partie des données métier spécifiques.

Base de données centralisée : Une base de données SQL centralisée, quantum-motors-db, est utilisée pour stocker les données de manière structurée et partagée entre les services.

Chaque micro service accède à la base de données pour effectuer des opérations de lecture et d'écriture de manière isolée, ce qui permet de maintenir la cohérence des données.

Gateway avec Traefik : Une passerelle Gateway utilisant Traefik est mise en place pour gérer le routage et le trafic entrant.

La gateway sert de point d'entrée unique pour le client Web externe, gérant les requêtes vers les différents services en fonction des chemins d'URL spécifiés (par exemple, /battery-service pour le service des batteries).

Le port source (externe) 8086 est exposé pour les connexions entrantes, qui sont ensuite redirigées vers le port destination (interne) 8080 de la gateway.

Trafic Réparti et Isolation : Traefik joue un rôle de reverse proxy et assure la gestion du NAT (Network Address Translation), permettant une meilleure distribution du trafic et l'isolement des services.

Cela permet aussi de sécuriser l'infrastructure en limitant l'accès direct aux services, tout en offrant une entrée centralisée pour les utilisateurs finaux.

Scalabilité et Indépendance : Chaque service peut être mis à l'échelle de manière indépendante, selon les besoins de l'application.

Cette modularité offre de la flexibilité dans le déploiement et la maintenance, et permet de répondre facilement aux pics de demande pour des services spécifiques.